Active InferAnt Stream 003.1 ~ Systems Within Systems: Blake, FOIA, Cog

ActiveInferAntStream | Systems Within Systems: Blake, FOIA, Cognitive Sovereignty, and PyMDP | 2:00:40

All right, welcome to Active Inference Stream number 3.1 on systems.

It's August 31st, 2024.

As usual, I'll begin it with fetching the origin, committing many, many updates, pushing them.

And while that's writing, here's what's going to happen.

We're going to be talking about systems within systems and also whatever else we end up exploring.

Touching upon some developments in the code base related to William Blake, FOIA, Freedom of Information Act, Cognitive Sovereignty, and the PyMDP, Social Learning.

We'll start it off by making an initial GitHub push.

I'll copy everything down below so we can delete from here.

GitHub push looks done.

Let's reload the repo, double check it's there.

And we see pushes now.

So we're triple checked live.

All right, made the GitHub push.

In this opening, the purpose of this stream, or at least one of them, is to explore various views on implementations of and interactions with systems.

Let's start by going through several code base developments that I've prepared to show.

Please, at any time, feel free to write in the live chat a question or a comment or idea.

So I'll go through these three and then another fourth section.

At that point, conclude.

Along the way or at the end, just write in the live chat if anything is like off with what you're seeing or if you have questions or ideas.

Okay.

Okay.

We are in folder zero, context, systems.

And we'll start in the William Blake folder.

We're going to look at, through some code methods, William Blake's mentions of systems.

Talk about some relevances for modern systems design and look at the applications in the code base.

Okay.

So within the William Blake folder, there are several scripts and there are several folders.

William Blake resources contains files like additional references, just bibliography, hundreds of relevant citations.

Blake's works by year and table of contents to Eerdman.

Then there's the Eerdman edited complete works of Blake and letters, which is 29,000 lines and the pumps, which is 8,200 lines.

As a reminder, in cursor, all of the code base is indexed, whatever the type is.

So there's nothing to resync here.
Let's double check though.
And I'm using cursor 0.40.3.
So 40.3.
Okay.
So a lot of ways to jump in.
The poems are great.
And this is also just the plain text.
This is not the illuminated printing.
I'll start with the Blake mentions.py.
Right click, open in the integrated terminal, Python 3, Blake mentions.py.
What this is going to do is look at the target terms in the top of the document system.
That's our focus.
But also just to show that it's a little more general.
Reason and los.
Logging is done.
And for each of the terms in the target terms.
Where the term is mentioned, ignoring case, three lines before and three lines after get joined into a snippet.
And that gets saved into an output file.
So here's across that document of Erdmann, which there can be typos.
There can be all other sorts of little textual artifacts.
Not everything's included.
Not all the works remained, etc.
But we can explore.
Okay.
These are the mentions of system in Blake.
There's nine of them from this poll.
Till a system was formed, etc.
I must create a system or be enslaved by another man's.
Striving with systems to deliver individuals from those systems.
Fixing their systems permanent by mathematic power.
To govern the evil by good and states abolish systems.
And here begins the system of moral virtue named Rahab.
No grand work can have them.
They produce system and monotony.
Before I begin engraving them.
As it will enable me also to regulate a system of working.
That will be uniform from beginning to end.

So that's kind of a cool way to jump in.
And noting that there's many more mentions of these characters, Erdmann and Los.
But same principle applies.
These are just much longer documents with snippets.
Often it's like reading through Blake.
There's all these entities.
All these things are being mentioned.
And so these two entity extraction methods use a simpler method.
Which is just basically looking for capitalized.
This is a regex.
That finds basically capitalized words.
Single or pairs.
Whether at a line start or not.
That start with capitalization.
And then it goes through and counts it.
The entity extraction with spacey uses this package for slightly more advanced entity recognition.
But in any case.
But in any case.
What these end up doing.
Is they output into the analysis section.
These plots are just example plots.
But we can see for example.
Here's a top 10 entities.
Here's entity frequency.
Histogram.
And so this counts the number of entities.
Of each time that the entities are mentioned.
So it kind of just sweeps through.
Finds the capitalized.
And the analysis is just doing some basic summary statistics on that.
So that's just kind of a fun way to jump in.
And explore how many entities are done.
And just striving with systems.
Okay.
That's part one on William Blake.
Let's go to social sciences and active inference.
Okay.
So.

This initially comes from Andrew Pachea's awesome work at the IC2S2 conference.
Where he made a workshop.
Active agents and active inference approach to agent-based modeling in the social sciences.
He made it in a Google CoLab notebook.
Which is awesome.
Very accessible.
And allows people to get active inference.
Multi-agent network learning simulations working in their browser.
That's awesome.
Notebooks are again.
Great for just one click running in a browser.
However.
They can get pretty unwieldy.
Because you have these linear documents that can sprawl on and on and on.
And it's hard to know what has been run and not and so on.
And so.
27 days ago.
I made a contribution.
Just to take his CoLab notebook.
And copy it into a single dot py.
So that just took his great.
As far as I understand.
Manually written code.
About 1500 lines of code.
And just brought it into a single Python script.
So it could be run as a script.
And then.
What I did.
Over the previous.
Day or two.
Having some fun conversations.
With Andrew.
In Discord.
Is.
Broke it up.
Across.
A bunch of different scripts.
So there's a main.py.

This is also.
It's more than just broken up.
There's also many other changes and developments.
Sometimes to the point.
Where it was like.
Just going.
Through.
Hands and minds.
And AIs.
And it's like.
Wait.
What is even happening.
But this was just a.
Again.
Experimental play.
Main.
Calls.
All the aspects.
Of the simulation.
And then.
The utilities are broken up.
Across.
Math.
Data.
Analysis.
Agents.
Creating them in the matrices.
And their networks.
Active inference.
Such as the active inference loop.
Itself.
And then the visualizations.
This is pretty cool.
Again.
It's a really helpful.
PyMDP styled way.
To work with.
What.

Andrew made.
Which is this awesome.
Multi-agent.
Communication.
Network simulation.
There's agents in sub networks.
And the output of this.
In the current output folder.
There's a couple methods.
That just visualize the matrices.
Of the different agents.
So.
A.
Matricy.
That's.
A for ambiguity.
That maps between observations.
And hidden states.
So here.
There's three observations.
And then there's three hidden states.
If it were perfect.
Sensory recognition.
You'd see only ones.
On the diagonals.
And zero elsewhere.
That's when the.
Partially observable.
Markov process.
Can be treated as a.
Fully observable.
Markov process.
B.
Is where action.
Comes into play.
That's where it becomes.
A Markov decision.
Process.

With a policy decision.
Around which slice of B.
To pick.
C.
Is a vector.
That describes preference.
That comes into play.
With the.
Pragmatic value.
In the expected free energy.
Calculations.
And D.
Is an initial.
Prior.
On.
Hidden state.
E.
Is the habit vector.
Which describes the prior probabilities.
On action selection.
F.
This is just showing the.
F at the very beginning.
I think it just given a tiny.
Value.
For the variational free energy.
Just to make sure it's all.
Initialized.
And G.
Not sure what's even.
Happening here.
That's for expected free energy.
For planning.
And.
Not that this was.
Tuned.
Or.
Even really.

Checked.
Super deeply.
For.
Concurrence with the initial.
Simulation.
But these are.
Fitness.
Of the solutions.
Of the agents.
Across two different.
Graph.
Topologies.
Here's a heat map.
Looking at the correlations.
Of some summary statistics.
Of these simulations.
There's.
Average fitness.
Max fitness.
And min.
Of the agents.
And then fitness improvements.
The variables have.
Of course.
Total correlation.
With themself.
But there's some.
Interesting patterns.
Here.
There is some.
Sweeping.
Across error rate.
And C.
Improvement.
Preference learning.
A.
After every.
X.

Time steps.
Distribution.
Of fitness improvement.
By network type.
Again.
Kind of interesting.
Possibly.
The case.
That the small world.
Network.
Leads to.
Better fitness.
Improvements.
Fitness.
Average fitness.
Over trials.
By network.
Type.
Here.
We see.
The random.
And the small world.
And their average fitness.
Changing.
I think this might reflect.
Why by the end.
Of the 160 trials.
They have an equivalent.
Average fitness.
However.
The small world.
Improves.
More.
Most improved.
Topology.
And let's see.
Max fitness.
Versus.

Min fitness.

Just a few.

Scatter data points.

Um.

The.

Simulations.

Extensively.

Logged.

And traced.

And.

It's just really cool.

Let's just try.

So that we can.

Explore a little bit.

On the interpretability side.

Control.

A.

Control.

Shift.

L.

That's going into the sidebar.

And we'll ask.

Comprehensively.

Explain.

This.

Simulation.

All the.

Components.

And methods.

All about.

How.

Exactly.

Active.

Inference.

Is being.

Applied.

Here.

This is the sidebar.

Chat.
One.
Thing.
I'm also out of fast.
Request.
So I really don't know.
How fast.
Cursor is going to process this.
Control.
I.
This.
Floating.
Composer window.
Is.
An incredible.
Development.
It.
Enables.
Pretty.
Easy.
Editing.
Across.
Multiple.
Files.
Which.
Can get really.
Far.
Ranging.
So it's definitely like a.
Check.
And find out what's happening.
But.
It.
Can do some amazing things.
And I think that.
Later.
In the stream.
When we get to more.

Code.

Editing.

We'll definitely be using.

The.

Composer.

A lot.

Okay.

The script sets up and runs a simulation.

Using active inference principles.

It involves initializing matrices.

Creating an agent.

Running inference loops.

And visualizing results.

The main.

Components include.

Logging.

Agent setup.

Running simulations.

And processing results.

We have.

Imports and logging.

Path setups and constants.

Initialization.

Initialization.

Fitness data initialization.

Visualization and inference loop.

Visualization.

Full sweep of the simulation.

Full sweep simulation.

Runs a full sweep simulation.

Across multiple trials.

Results processing.

Final visualization.

Active inference application.

Active inference is applied through the following steps.

One.

Agent initialization.

The agent is initialized with matrices that represent its beliefs about the world.

That's the joint distribution.
Agents are mapped.
Represented.
Compositional cognitive cartography.
About their beliefs.
About observations.
Hidden states.
And policies.
Action selection.
The world.
The agent runs an inference loop.
Where it updates its beliefs and actions.
Based on observations and internal models.
Three.
Learning and adaptation.
The agent adapts its internal models.
As matrices.
Based on the learning parameters and observations.
And four.
Simulation and visualization.
And now it's also pulling in real links to PLOS.
For example here.
And a medium tutorial.
That is Oleg's medium tutorial from 2018 on active inference.
Cool.
Okay.
Let's just go a little bit further.
Here we're using GPT 4.0.
We can ask.
How does the specific implementation of active inference here in utils underscore?
Active inference.
Differ.
From other agent based models.
Give.
Comprehensive.
Concise.
Answer.
Also.

Conveying.

Why slash how slash where slash when.

Quote.

Free.

Energy.

Calculations.

Are.

Specifically.

Coming into play.

It was a great question.

Someone brought up in the institute discord.

Pretty recently about.

Well.

How does active inference differ from embodiments?

Haven't people been thinking about perception action in terms of their unity for a long time?

Let's kind of ask a variant of that question.

Implementation of active inference in the utils script differs from other agent based models primarily in the use of the free energy principle to guide agent behavior.

Here's a comprehensive yet concise explanation.

Key differences from other agent based models.

One.

Free energy minimization.

Agents minimize a quantity called free energy which is a measure of the difference between predicted and actual sensory inputs.

This drives both perception.

Updating belief.

Change your mind.

And action.

Selecting actions to minimize future free energy.

Expected free energy.

Change the world.

Other models.

Us.

Them.

Other models.

Traditional agent based models often use predefined rules, heuristics, or reinforcement learning to guide agent behavior.

Two.

Regarding belief updating.

In active inference, agents are continuously updating their beliefs using Bayesian inference.

Integrating new sensory data to minimize the variational free energy.

Other models.

Belief updating may not be as central or they may use different mechanisms like state transitions or key learning.

Three.

Action selection.

In active inference, actions are selected to minimize expected free energy.

G.

Which includes both the expected, surprise, and uncertainty about future states.

Equation 2.6.

In the 2022 textbook.

Other models.

Actions are often selected based on maximizing rewards or following predefined strategies.

Cool.

Nice.

Free energy calculations and active inference.

Where and how free energy is calculated.

One.

Agent initialization.

That's the A, B, C, D, E.

Here's action sampling.

Agents sample actions based on their current beliefs and the goal of minimizing expected free energy.

Free energy update.

Agent.update free energy.

Agents update their free energy based on the new observation and actions taken.

And during each time step, the script tracks various metrics including average expected free energy, av.e.f.e. list and variational free energy, av.e.f.e. list.

Why free energy is important?

Oh wow.

Perception and action.

Free energy unifies perception and action under a single framework.

By minimizing free energy, agents improve their models of the world and select actions that reduce uncertainty and prediction error.

Adaptation.

This approach allows agents to adapt to changing environments dynamically as they continuously update their beliefs and actions to minimize free energy and some code.

Comparison model models.

Reinforcement learning focuses on maximizing cumulative reward.

Rule-based models use predefined rules.

Evolutionary algorithms use population-based search methods, which Nassim and I recently have explored in the traveling salesperson problem in the ant colony optimization.

So, pretty cool.

Thanks Andrew for making the original open source educational materials.

Let's go a little bit deeper into PyMDP poll.

Okay, here we're in systems active inference.

PyMDP poll.

It's a super fun script.

I hope that PyMDP learners, respecters out there, give it a try.

Let's open it in the integrated terminal.

Python 3, PyMDP poll.

Here's what this does.

First, it clones into the specific repo of PyMDP.

Then, it lists all the methods, and it rolls through all the methods in the submodules, and it writes it to PyMDP methods.markdown.

So, this is a markdown file that is, let's see, 2,500 lines long.

But how cool.

These are all the PyMDP methods listed.

Documentation really matters.

Here's the arguments for PyMDP.agent, which is in the agent submodule.

So, here we can see the full arguments, the arity of the agent, and example usage.

This is not being generated by LLM.

This is just being pulled from the doc strings and the information that's in the PyMDP package.

So, again, those contributions, the questions that people ask, really do matter.

Here's PyMDP algorithms run MMP.

That's marginal message passing for updating marginal posterior beliefs.

It might be buried somewhere, who knows where, in a script, and here it is brought out for all to see in terms of the parameters that are used and what it returns.

So, it's like, hmm, wow, this is really a cool way to study and learn any package, but here just in the PyMDP setting, like 1,000 lines later.

PyMDP.control.updatePosteriorPoliciesFullFactorized.

Here we see, okay, we have ABC, A-factor, all these arguments.

What does it do?

And this is the doc string, not LLM generated.

But this is awesome for helping read what the doc strings are, not by scanning through and seeing the plain text of the code, but rather by being able to look and say, like, hmm, okay, what were the comments written?

But with PyMDP, we could also pretend for a second like we are going to use some LLMs for development.

Here, we'll use Command-I.

We'll say, update and professionalize these methods without altering function.

For example, with comprehensive doc strings, we're missing.

Andrew, in a live chat, wrote, amazing work, Daniel.

Thank you for presenting my work and, more importantly, expanding on the original goal, open source learning, open source resources.

Totally.

Totally.

Okay.

So, here, we're going to update, and then we'll put in, like, a special word or just kind of like a weird symbol.

But if we ran the PyMDP poll again, it would pull the whole PyMDP cleanly.

So, it won't include our updates.

Also, cursor doesn't, at this time, do very well.

It says above 400 lines.

Certainly, it gives mixed results.

Once you get, like, over 500 or 1,000 lines, it does give mixed results.

For example, sometimes it'll do a fix, like, up near the top of the document.

Okay, we're about to find out.

Dun-dun-dun-dun-dun-dun-dun-dun-dun.

It'll do a fix up at the top, and then it'll be like, okay, and then delete the rest.

So, here, it's just showing this is, like, LLM augmented development on PyMDP.

Now, this is funny because what it did for the doc strings to the underlying functions was it peeped into the summary we already had.

And now it's reentering the doc string in terms of what it has for parameters and returns.

So, again, not saying that's the right way to update, but this starts to get into looking really cool.

Like, oh, maybe there could be, like, unified formats for what the doc strings are,

and we could use the active inference ontology in a really coherent way

and have the inputs and outputs and the arguments of each of the functions.

That's important for translation learning and for our accessibilities, also for machine sense making.

So, pretty cool.

I'm going to delete the PyMDP repo version here,

and that's where we got to with PyMDP poll.py and studying the PyMDP library.

Okay.

Into part three.

Cognitive sovereignty.

Okay.

So,

Cognitive Sovereignty and Active Inference in the State of Exception.
This was a paper that I wrote in October last year,
and the paper provides an analysis of Giorgio Agamben's book,
Homo Sacre, in the tradition of active inference.
I'll just read this first paragraph.
Homo Sacre articulates the relationship between bear life and political existence
in Western politics and metaphysics.
Agamben argues that politics is founded on the inclusive exclusion of bear life,
where natural biological life, the physiology and cognition of the body,
but reading later and more, I understood it's not simply only that,
is politicized only through its exclusion as an exception.
Drawing on Aristotle's definition of man as a political animal,
Agamben traces the historical development of this structure
and its continuation in modern biopolitics.
So, in this paper, which was super fun to write,
and I got helpful feedback from colleagues as well,
first, I reviewed my reading of Homo Sacre
and some of the logic of sovereignty,
drew an illusion between normal politics and the state of exception
to normal and revolutionary science in the Kuhnian paradigm,
then played off this kind of fully synthetic concept,
synthetic intelligence, to explore cognitive sovereignty,
and then considered a situation that we were modeling
where active inference agents are having a normal political scenario,
then there's a crisis which induces a state of exception.
And then, three terms from the active inference ontology,
but this is kind of a Mad Lib plug-and-play,
affordances and how that changes during the state of exception,
variational free energy in terms of bare life,
and expected free energy in terms of the sovereign's agency
during the state of exception.
And then, wrote an active state inference model,
sovereign agency in the state of exception,
and gave some pseudo code.
So, it was a fun paper to write,
reviewed some of this humanities and political sciences scholarship,
and then got to some pseudo code.
Earlier LLMs.

Oh, October 2023.
And then, okay, what could we do next?
Well, we could include a bunch of other topics,
and then we could also, like, actually make functions.
We could make the functions that are described,
and we could go from active state inference,
the sketch above, to active governance.
For example, multi-agent setting, cognitive security,
quantum cognitive sovereignty,
all these kinds of fun topics.
So, following a recipe,
a meta-recipe,
that we've seen many, many times.
First, what I did is,
I brought in the cognitive sovereignty paper.
So, I just copied in the full paper.
280 lines.
Because one line is like a whole paragraph.
It's about the number of tokens anyway,
but the new line thing,
this is kind of a cool feature.
Then, I basically just did
control A for all,
control K,
and then I prompted,
I said, okay,
write a specification
for what the model actually is.
And so, it describes,
okay, well, you describe the state space.
This is kind of like chapter 6
from the 2022 textbooks.
What is the hierarchical nature, if any?
How do we think about precision?
On what?
Variational free energy,
expected free energy.
What is the state of exception?

What is triggering the state of exception?
How do we build a generative model,
generative process?
What algorithms do we need?
How do we think about the multi-agent setting?
How do we think about cognitive security?
And then, how do we think optionally
about quantum exceptions and extensions?
Scenario generation, validation,
ethical considerations.
So, this is a technical spec
that provides a comprehensive blueprint
for implementing
an active inference generative model
to study cognitive sovereignty
and states of exception.
It incorporates key concepts
from the original paper
while providing concrete implementation details
and extensions.
Then, I kind of asked it to go
one more step
and said,
make a technical prospectus
for this project.
And again,
this is a smaller,
but pretty similar.
So, those resources are there.
And then,
just go to the gear
and then features,
resync.
I don't know how often
it resyncs by itself,
but it's just good to know.
Okay, we're all caught up.
Close.

Close term.
Okay.
Now, let's open up the script.
So,
main.py
is going to do
some pretty cool stuff.
And these are like meta patterns
that are just so useful.
There's a config.json
that has the parameters.
Here,
it's just the input and output directories
and then how many time steps
before the crisis
are we going to simulate
and then how many time steps
after the crisis
are we going to simulate.
There's a utilities
that contains
the different kinds of class
and function definitions.
And then,
main is going to call
in three steps,
also printing each line
of the output in real time,
main is going to call
one, two, three.
So,
first,
it's going to run
one,
generate an entity library.
We'll look at the structure of that.
Two,
it's going to run

the simulation
according to the config.
And then,
three,
it's going to perform
a simulation analysis
also based upon the config.
But I don't think
there's any analysis
free parameters,
but it could be.
So,
Python 3
in the terminal,
main.py,
running one.
So,
let's look at the outputs
to terminal.
Generating entity library
for cognitive sovereignty simulation.
It saves
an entity library.json
to the output folder.
We'll look at through these.
There's 15 entities.
There's
five kinds of entities.
We got government,
corporation,
NGOs,
media,
and citizen groups.
And then,
we have
entity one,
two,
three for each

type.
It starts out
where government one
is in power.
All the others
are not in power.
Okay.
Now,
it's running
the simulation.
The simulation
actually runs
like super fast,
but it takes
a few seconds
to plot
one of the
figures.
Let's just look
at the entity library
while it's running.
Okay.
So,
here's how each
entity is described.
These are
well-spaced
matrices
that describe
the transitions
amongst
five different
states.
The five
different states
are
in power.
So,

that's when you
start or when you
gain power.
Then,
there's three
kind of
continuously
varying states.
This is just a five-state
discrete model.
high,
medium,
and low
power.
So,
this is kind of
like close
to being in
power,
medium power,
and then
low power.
And then,
there's
homo sacer,
which is
the category
for that
which can be
killed with
impunity
explored in
the book
of the same
name.
So,
here we go.
Let's look back

to the terminal,
then we'll look
over all the
other outputs.
Okay.
Ran the
simulation.
Visualize.
This is the
step that takes
the longest
just by
visualization
methods,
but it's a
big figure.
Entity traces
are recorded,
then the
simulation analysis
is wrong.
So,
okay.
Let's look at
the analysis
plain text
summary,
but it'll
get more
colorful.
So,
we see,
okay,
state volatility.
These are
just volatility
levels.
Like,

government one
has the
lowest
variance
through time,
probably because
for the
first 200
time steps,
it just
stayed in
power,
whereas the
other ones
were bouncing
around during
phase one.
Here,
most common
states.
So,
let's see.
Here's the
most common
states before
and after
the crisis.
Like,
corporation one
was most
commonly in
low power,
but then
afterwards,
it was most
commonly in
the homo sacre
state,

or so on.
And then
this top
part,
state percentages.
So,
here's
corporation
two.
Here's it
before it
was bouncing
between high,
medium,
and low
power.
These are
just random
matrices,
so it's
not super
surprising that
they're roughly
being drawn,
but here we
see like
citizen group
one was
spending more
of its time
in a low
power situation,
but then
after the
crisis,
it was
spending,
you know,

18% of the
time in the
high power,
39 in the
homo sacre.
Okay,
here's the
traces of
the visualization.
So,
it's a little
bit condensed,
we could run
one for like
10 time steps
if you want to
see the
bouncing around.
Oh,
but very
interestingly,
this could be a
slightly different
one,
actually that
lineup,
so that was
the government
one got
displaced in
the crisis,
some new
group,
maybe that's
media two,
but it just
happens to be a
very similar

color,
but it is a
different one
that comes in.
Here's the
results of the
simulation,
many,
many thousands
of lines.
Here we see
the overall
occupancy of the
different states.
So,
we had one
out of 15
were in
power before
and one
out of 15
stayed in
power after,
it's a different
one.
And then,
here's like the
introduction,
in this kind
of toy
situation,
there was no
homo sacre
before the
crisis,
but then it
opened up like
this niche.

State
percentages,
the visualizations
are a little
bit fun.
They have
their own
charm to
them,
but here we
see like
government one
was spending
100% of its
time before
in power,
and then after
it has,
but these are
overlapping bar
charts,
that could be
just visually
clarified,
but we can
see like which
one was
most,
like media
two was
spending 100%
of the power
after.
Okay,
this is the
big figure.
So,
each row is

an entity,
and then let's
look at these
B matrices.
So,
here,
all start
out at
baseline,
and government
one is just
given the
power initially,
but at
baseline,
you stay
in homo
sacre if
you're already
there,
but no one
starts there,
so no one
is there
initially,
and you
stay in
power if
you're already
there.
Again,
same thing,
one is just
given it.
So,
the others
that aren't
in power

are just
bouncing around
in this
sub-matrix
between high,
medium,
and low.
So,
these are
transition
matrices,
just Markov
chain
processes.
Then,
once the
crisis happens,
there's two
kinds of
states.
There's
empowered
and disempowered.
So,
for all
entities,
once you're
empowered,
there's no
secondary changes
in power
structure in
this simulation.
Once you're
in power,
you stay
there.
When you're

empowered,
you just
stay in
power.
When you're
disempowered,
now,
you can,
this is
from
to
to.
Once you're
disempowered,
so that means
you're not
the sovereign
after the
state of
exception has
begun.
From
to,
so here,
if you
are,
if you
happen to
just,
for that
first time
step,
be in
power,
then you
get kicked
to high
power with

one.
That's what
they all
have that
one there.
And then,
now you're
exploring this
four sub
matrix amongst
high,
medium,
low,
and
homo
sager.
So,
kind of
cool.
Gives a
starting point
and,
and,
we could
possibly
fuse that
with
pi-MDP
because in
the paper,
the definition
of sovereignty,
cognitive sovereignty,
was for
one's own
EFE to
influence the
VFE of

others.
So,
when the
political
operativities
of one
agent
influence the
bare life
of another,
that's the
state of
exception.
That's
political
agency.
So,
that was the
cognitive
sovereignty
simulation.
simulation,
it's like
cursor and
the LLM
probably could
have gotten
there a
year ago,
a little bit
less than a
year ago.
So,
it's like,
not that this
simulation was
implausible,
it's just

accelerating more
and more
with going
from,
even the
pseudocode,
I don't know
how much it
was needed
because it
was itself
just sort
of giving
a few
sketches.
So,
cool.
Okay.
We'll
possibly
return and
add PyMDP.
Like,
meanwhile,
if anyone
in the
live chat
has a
request or
a thought,
otherwise,
we'll look
at this
fourth section
and then
we'll
recap.
Look at

all these
suggestions.
Like,
did you
mean
eternal
return?
Is that
what you
meant?
Okay.
FOIA.
Let's
bring in
a kind
of
realist
twist.
So,
in
other,
there's
all these
different
frameworks
and
different
kinds
of
systems,
hence
systems
within
systems.
And
let's
go to
FOIA.

All right.
United
States
Freedom of
Information
Act,
FOIA,
this was
generated
material,
is a
pivotal
federal
statute
enacted
1966,
designed
to ensure
transparency
and accountability
in government
operations by
granting public
access to
federal agency
records.
Key technical
details include
its enactment
purpose,
scope,
request process,
exemptions,
appeals,
litigation,
fees,
electronic
FOIA,

there was a
1996 e-FOIA
amendment
mandating
the electronic
availability of
certain records
in the
establishment of
electronic
reading rooms,
whoa,
annual
reports.
FOIA is
an essential
mechanism for
fostering government
transparency and
accountability,
enabling the
public to
remain informed
about governmental
actions and
decisions.
Okay,
so there's a
few different
scripts in
here.
There's a
main.py.
This is just
a wrapper
that's going to
call several
of these other

scripts,
but let's
look through
them.
So,
FOIA
utils.
First,
it loads an
API key from
the .env
file.
So,
as brought
up before
in the
context of
LLM APIs,
git ignore
is at the
top level
of the
repo.
And here,
you can
specify
types of
files,
like file
extensions,
and also
whole sub
folders.
Like,
when we
clone in
other big
repos,

we don't
need to
then push
those as
if they
were kind
of ours.
That kind
of breaks
the git
chain.
But we
see .env
here.
So,
what happens
in
utils
is it's
going to
load in
using the
.env
Python package
the API
key.
FOIA
API
key.
So,
each person
is going
to get
in their
API
key,
and mine
is in

here.
I'm not
going to
show it
on the
stream.
But then,
when you
make your
API request,
it will
use your
API key,
and then
you can
keep
your .env
with your
secret keys
for LLMs
or for
the government
or whatever
it is,
and you
know that
it's not
going to
get pushed
to GitHub
like when
you do
a push.
Utils
has
different
utilities.
Here are

some example
FOIA calls
that are
just like
more
formulaic.
There's an
analysis
and a
markdown
translator.
And then
agency year
has a
start and
an end
year.
So here
we did
2000 to
2025,
but I
also
explored
doing
like
1200
to
2400,
just
thousands
of years
that we
wouldn't
expect,
but you
never know.
And then

also about
a few days
ago when I
initially did
this,
I got
rate
limited.
So I
had to
have some
email
correspondences
to restore
my access,
but that
itself was
fun and
interesting.
Then I
used LLM,
I just
would copy
this,
let's just
see,
shift here,
just selecting
this,
control K,
just to
bring in
just that
part,
add any
missing
agencies.
Okay,

let's just
see if,
and I
just did
that several
times just
to get
a bunch
of agencies.
I don't
even know,
these may
not even
be real.
Let's see
if any
are added
here.
Okay,
more
agencies.
It's
like,
okay,
now,
now you've
gone too
far.
Let's
just see
really quick.
USDA
APHIS,
this sounds
agricultural.
Yeah.
Animal
and Plant

Health
Inspection
Service,
that's real,
kind of like
APHID.
Some of
these,
it's like,
really though?
see that
JMMFL
doesn't seem
to be an
actual one,
but regardless,
that's just
going to
still get
called.
So,
again,
it takes
the start
and end
year and
all the
departments
listed.
We could
do a
hashtag
out or
we could
change all
agencies.
We could
say list

of just
and then
just delete
that if
we only
wanted
major
departments.
Or,
of course,
you could
just delete
the ones
you don't
want and
then just
say,
okay,
only interested
in NSF
for this
year.
Okay.
It sorts
it alphabetically,
converts it
to dictionary,
and then it
fetches this
utility.
If we were,
you know,
but this is a
common move,
so I might as
well just
show it.
Select

all,
and then
did I,
but it
wasn't really
needed because
even just
pulling up
I,
control I
with the
composer,
it already
brought in
this script.
Then,
I'm just
going to,
again,
just for
doubling
check,
add into
the context
window foia
utils.
Move
all the
methods
defined in
at
foia
underscore
agency
year
to
foia
underscore

utils,
and
invoke
them
properly.
Here.
So,
expected
behavior
would be
it's going
to delete,
red lines
are going
to show
up from
93 to
108,
and then
we'll be
able to
check.
And again,
this is why
being able
to,
here it
is,
it's
drafting
on these
two
subtabs.
And then,
let's see
how it
goes.
So,

whoa.
So,
but it
can go
too far.
So,
it may,
I'm not
going to
accept this
edit this
time,
just because
I know
it's already
working.
This looks
like it
might have
gone too
far.
but if
it's a,
um,
what it
usually does
or what
you can
get it
to prompt
to do
is just
pull
methods
that are
defined
with def,
pull them

into utils.
Like,
what it
would manually
look like
would be
copying in
control x,
cutting out
def,
putting that
to foia utils.
and then
ensuring that
the,
um,
foia agency
year from
foia utils
is pulling in
fetch and save
annual reports.
that'll still
work.
Okay,
so what,
what is this
output?
Clear the
terminal.
We're still in
cognitive sovereignty.
I'm just
going to
trash that
terminal.
Right click.
Open an

integrated
terminal.
Python 3.
Let's just
do main.
Okay,
so main
we did an
update,
but again,
this is what
it looks like
to,
to debug.
Just put it
into the
composer.
Fix it.
Meanwhile,
let's call
the,
the foia agency
year.
Accept it.
Again,
looking is
probably a
good thing
to do.
It's like,
it's not just
like not
looking before
crossing the
street.
It's like
not looking
when making

stochastic
edits to
your PhD
dissertation,
which is
live code,
but
c'est la vie.
Okay,
there we go.
So even
without intervention,
foia agency
year.
Okay,
so what it's
doing is
it,
maybe we're
getting rate
limited,
I'm not
sure.
For each
of those
departments,
it's saving
these outputs.
So we'll
keep watching
what's
happening
live.
I'm not
sure maybe
this is
going to
be about

to be a
huge one.
But first
it checks
if we
already have
that exact
combination
of agency
and year.
Then I
concatenate in
the file name,
this is just
one way to
do it,
it could be
done like
way better
ways.
The date
that it
was pulled,
that's
2024,
8,
20.
I'm not
sure if
that last
piece is
the time,
but then
the middle
piece here
is like
19
characters.

Because for
this one,
like 2000
DOI,
let's just
look at
the USDA.
Okay,
so 2005,
it's 19
characters long,
you're going
to see a
lot of
19s,
because 19
just says
report not
found.
So if you
request,
what was the
annual report
for the USDA
in like the
year 30?
3400,
probably it's
not found
on that
server.
Not saying
it's not
there,
but then we
can see,
okay,
here's 2010

annual report.
Okay,
interesting,
but we
got it.
So this
is their
FOIA
metadata.
As far
as I
totally
understand,
this is
all open
information.
I am
not doing
anything other
than just
calling the
FOIA API.
so we
can see
how many
characters
are there.
And then
it's like
2024 and
2025 are
back to
19 characters
because those
reports are
not there.
Not sure,
like it'll be

interesting to
see what
comes out
here.
clear.
Okay,
but I'm
just gonna
kill it.
Clear.
And then
let's
call
FOIA
to
markdown.
So here,
it's gonna
make a
FOIA
markdown
folder.
So it's
just iterating
over all
the reports.
It's giving
a lot of
errors.
Maybe we
could debug
it,
but we'll
see.
Let's go
DOE.
So here's
the 19

character
one,
report
not found.
But here's
the 43.
Okay,
now this
is looking
a little
bit more
readable.
Let's
make a
new script
called
FOIA
Secondary
MD
Processing
dot
py.
It's
already in
the context
window.
I'll add
into the
window
a
markdown
that we
know has
a lot
of
characters.
So we'll
do DOE

2008.
Let's try
without
specific
reference
to a
file.
It would
work with
reference to
a specific
file,
but just
make sure
that it
doesn't
overfit
there.
Let's
say
given
markdown
documents
in
dot
slash
that means
this
folder
FOIA
underscore
markdown
slash
folder
and all
sub
folders
write

secondary
processing
algorithm
that
reads
in
and
improves
formatting
and
conciseness
saving
all
outputs
to a
folder
dot
slash
FOIA
secondary
markdowns
slash
open
parentheses
make
it
if
it
is
not
there
recapitulating
folder
nested
topology
thank
you
I'm

curious
to see
because
it's
not
going
to
do
an
LLM
call
but
if
you'll
recall
back
to
infra
ant
stream
one
and
the
kinds
of
LLM
API
augmentations
that were
possible
then
we
could
imagine
okay
now
iterate
over

all
these
reports
and do
LLM
summary
or do
translation
or explain
it to
this
person
okay
let's
just
see
how
it
goes
all
right
clear
we're
in the
FOIA
folder
here's
secondary
MD
processing
Python
3
FOIA
tab
it
goes
to
the

point
where
it's
ambiguous
and
then
secondary
MD
processing
okay
no
module
BS4
some
people
might
know
ways
to
fix
this
but
let's
just
see
what
it
recommends
okay
it
adds
some
I'll
just
accept
it
clear
run

it
again
error
beautiful
soup
4
is
not
installed
installing
it
now
so
this
is
what's
so
fun
and
collaborative
is
like
oh
okay
I
came
across
that
friction
and
then
just
within
a few
seconds
okay
but
now

no
one
else
will
hit
that
exact
friction
okay
interesting
so
let's
look at
what
it's
doing
whoa
okay
this is
processing
from ones
that are
that are
like
just
kind
of
empty
in
so
we'll
see
what
those
maybe
those
have
a

different
format
but
it'll
get
through
them
okay
it just
looks like
it's
spacing
it out
but
you know
what's
wrong
with
that
and
then
let's
see
when it
goes
through
the
sub
folders
or what
happens
then
okay
it's
almost
done
with
the

ones
that
were
just
in
the
folder
itself
meanwhile
this
is
the
last
section
that
I
had
prepared
to
go
through
so
please
write
any
comments
or
questions
or
ideas
in
the
live
chat
and
then
we
can

play
around
a little
bit
and
do
a
little
more
developments
okay
here
we
go
OIP
okay
okay
okay
okay
not
okay
OIP
might be
an
exception
okay
let's
see
another
one
HUD
okay
so
19
okay
this
is
the

report
not
found
and
here's
okay
okay
okay
you know
is this
helpful
possibly
would
LLM
being used
in this
loop
be helpful
yeah
probably
okay
just
pull this
off screen
for a
split second
but
API.data.gov
I mean
this is
what
USA
OS
is
like
when we
think
about

what
what
what is
USA
OS
well
interestingly
I'm getting
some
very strange
internet
patterns
from the
other
window
which I'm
not going
to show
but
let's
just
end
that
one
and
see
if
I
can
pull
up
USA
OS
wow
very
interesting
here
I've

just
navigated
over
here
but
here's
from
my
personal
page
drawing
from
Kirby
Erner
USA
OS
but
it's
like
has
it
ever
been
more
API
oriented
to
interface
with
speak
program
utilize
the
USA
operating
system
not
sure

okay
clear it
close the
terminal
I'm going to
delete the
markdowns
because those
can be
regenerated
but I'll
leave the
responses
that already
came in
just because
those are
information
it's an API
call that
anyone could
have made
I'm just
listing them
there
so
that was
the FOIA
okay
alright
alright
alright
okay
let's
recap
there was a
couple
sections here

going from
the
end
to the
first
most
recently
we looked
at this
freedom of
information
act
API
call
scraping
processing
handling
pathway
and
explored
with the
git
ignore
you can
put your
own
personal
API
key
into
.env
you need
an email
address
to sign
up
and then
that's a way

where like
many people
can collaborate
on the
open source
tooling
while also
keeping their
API keys
secret
that's the
FOIA
then
we looked
at
in
folder
nine
other
and then
we looked
at
two
different
well
I guess
three
folders
within
zero
context
systems
now
here
going
in
forward
chronological

order
we looked
at
William
Blake's
work
entity
extraction
some
processing
and some
snippets
from
Blake
for
augmented
Blake
France
looked
at
cognitive
sovereignty
skipping
to the
third
one
of the
first
three
talked
about
how
this
meta
recipe
for
bringing
in a

resource
folder
asking
it to
write a
technical
spec
or a
prospectus
and then
writing
software
that's
very well
structured
and articulated
for further
development
with utilities
and mains
and configs
and how that
resulted in
some pretty
interesting
outputs
like just
about
looking about
transition
matrices
and setting
us up
to integrate
with PyMDP
which we
very well
could do

if someone
mentions it
in the
live chat
in the
coming minutes
and then
lastly
in the
active
inference
system
section
the
PyMDP
pull
method
and I'll
leave the
PyMDP
methods
up
let's
just
intra
stream
003
push it
and then
let's read
PyMDP
methods
in its
natural
context
that was
one hour
ago

reload
the page
reload
the page
now it
says now
zero
context
systems
active
inference
PyMDP
methods
this is
like
getting
into
oh
wow
this is
like
kind
of
nicely
formatted
this is
like
something
you could
take
on a
nice
trip
okay
Andrew
is writing
a few
things in

the chat
I'll
wait a few
seconds while
you type
out some
more ideas
anyone else
though again
please write
any ideas
and for a
couple of
minutes here
let's just
chill
see what else
we can
explore
and learn
and do
actually
while people
are writing
I'm just
going to do
a quick
overview
on what
is in
these
different
folders
so
starting
in
zero
computer

languages
this is
a language
name
brainfuck
this is
a brainfuck
implementation
of active
inference
famously
an obscure
arcane
bizarre
coding language
but
why not
right
it's all
just semantics
then
we have
brainfuck
active
inference
golang
thing
agent
definition
java
active
inference
pearl
julia
in julia
it's not
done yet
but

kobus
e has
done
incredible
work on
visualization
and many
many other
things
and so
I was
working to
get a
script
working
that would
render
the kinds of
images that
he's doing
and do a bunch
of other
stuff but
that's just
in progress
rust
and then
like hilarious
stuff like
active
shellference
I mean
this is a
shell script
that does
active
inference
this is

really
highlighting
the
semantics
and the
syntax
and the
space
between
and the
kinds of
pole
jumping
that we
can do
back and
forth
specs
and prompts
some early
documents
meta
informants
bio
informant
systems
folder
we have
active
inference
cognitive
sovereignty
IC2S2
and William
Blake
we went
into
today

P3IF
and some
of the
bolts
business
legal
operating
technical
social
these were
explored
in some
other streams
about
legal
engineering
and the
three
R's
and all
of that
so that's
the
zero
context
folder
that's
setting
context
here's
in
prepare
this is
a fun
folder
it has
a ton
of

it has
config
with all
these
configurations
for an
ant
active
inference
multi-agent
simulation
then there's
meta
config
that
configs
the
config
and then
there's
meta
meta
config
that
configures
the
meta
config
we have
cognitive
security
cryptography
digital
twin
design
processing
digital
twin

metaphysics
metaphysics
spec
generator
this is the
speculative
realism
coding
at play
nested
systems
systems
within
systems
invoking
the category
theory
networking
pheromones
the pheromone
ontology
quantum
pheromone
environment
and then
a little
bit of
culinary
delight
with the
salt
fat
acid
heat
pomdp
methods
this is
where

there's a
variety
of methods
that are
developed
with
LLMs
research
methods
learning
education
also
please
contribute
to
everything
and then
things
here's
different
things
including
a
generic
generic
generic
generic
logged
constructed
thing
context
again
is just
sort of
like
a bunch
of
different

things
that
contextualize
for us
and for
Shirley
cursor
one
is
preparing
with
configs
and
things
definition
and
methods
definition
then
two
goes
into
operate
here
is
planning
rendering
a plan
executing
the rendered
plan
of a
simulation
and
having
situational
ant
awareness

along
the way
the actual
operativity
of the
simulation
then we
have
measurements
measuring
the
simulation
four
reporting
on the
measurement
of the
simulation
five
following
up
and
specifying
and
executing
the
follow-up
on the
reporting
of the
simulation
six
is an
API
for
meta
and
format

for
knowledge
I
don't
have
too
much
in
here
seven
and
eight
are
not
there
and
then
nine
is
this
meta
pattern
where
we
clone
in
the
github
repos
or
the
github
users
for
a
given
domain
like

the
brain
these
the
brain
API
and
then
bring
them
in
reindex
cursor
then
program
and
then
we
can
delete
the
original
repos
so
that
that's
a fun
way
for
if
there's
something
people
want
to
integrate
there's
a few

multi-agent
LLM
techniques
and
AGI
related
things
that
would
be
pretty
interesting
maybe
for
a
future
one
so
that's
the
active
inference
package
okay
thank you
Andrew
I'm going to
copy what
you wrote
in
okay
let's
go back
to
0
systems
3.1
okay

let's
just do
003.2
.md
as if
there was
the .2
okay
Andrew
wrote
quick thought
on
actinf
pymdp
cursor
ai
integration
flow
one
build
out
working
scripts
plus
add
to
repo
eg
example
social
sciences
two
cursor
reads
working
script
repo
to

cursor
help
as
much
as
possible
to
finish
the
simulation
do
manually
what
it
can
cannot
its
horizon
3.
rinse
and
repeat
we
hit
the
limits
of
the
currently
existing
LLMs
along
the way
but we
build
the
repo

and
available
academic
simulations
resources
as we
go
while
improving
the
repo's
ability
to
provide
cursor
with
useful
information
for
improving
next
script
building
iteration
!
I
believe

this
is
what
you're
proposing
just
exercising
my own
understanding
of it

here
smiley
face
okay
thank you
Andrew
I'm going to
delete that
last one
now
let us
do this
control
a
control
k
operationalize
this
into
a
comprehensive
development
plan
for
what
I
will
immediately
do
after
your
response
comment
l-l-m
I
don't
know
if it's

been
explored
whether
saying
things
like
thank
you
or
speaking
to
it
a
certain
way
help
awesome
okay
okay
okay
make a
new
folder
called
um
augmented
active
inference
okay
let's
just
try
this
here
we're
in
3.2
.md

so
we
just
took
a few
super
chats
from
Andrew
and
spec
it
out
okay
so
create
a new
github
repo
doesn't
need
to be
a
github
repo
build
working
scripts
cursor
AI
integration
documentation
and
testing
iteration
improvement
next
steps

continuous
integration
okay
control
A
control
I
it
wasn't
necessary
it's
just
bring
it's
like
the
whole
file
and
lines
1
through
62
but
that's
the
whole
thing
but
it's
just
it's
a
habit
what
can
I
say

do
all
of
this
making
all
the
files
needed
saving
saving
the
new
files
as
created
in
.slash
augmented
active
inference
okay
let's
see
certainly
okay
it
looks
like
it's
doing
that
in
the
top
level
repo
of

the
whole
inference
but
we'll
bring
this
in
and
we'll
just
replace
this
folder
that
we
made
okay
drag
in
here
we
go
okay
reset
reset
this
is
just
because
I
I
I
rug
pulled
where
it
had

written
it
and
and
moved
it
okay
here
we
go
okay
script
overview
social
science
sim
dot
py
implements
a basic
social
science
simulation
using
Pymdp
okay
save
it
close
it
social
science
okay
from
Pymdp
import
agent
inference

learning
social
agents
okay
I'm
going to
run
py
mdp
pull
just
so
that we
have the
actual
py
mdp
repo
actually
let's
see if
we
don't
need
that
first
okay
we're
in
the
augmented
okay
social
agent
run
simulation
okay
utils

dot
py
process
visualize
calculate
tests
cool
unit
testing
all right
open it
with integrated
terminals
so we're all
the way
deep
we're deep
into this
cursor
augmented
situation
python
3
social
science
sim
okay
nothing
happened
so
here
control
i
add
extensive
logging
here
and in

the
whole
just
here
so we
can have
better
understanding
and
achieve
in this
folder
the aims
of
003.2
fully
okay
certainly
i'll
enhance
the
logging
in the
social
science
sim
dot
py
script
to
provide
a
more
comprehensive
oh
okay
okay
looks

good
now
add
the
utils
into
the
context
probably
not
necessary
probably
not
necessary
but
move
all
methods
to
at
utils
dot
py
and
invoke
them
here
that's
what i
was trying
to do
earlier
so let's
just see
what it
does
now
so

it
deleted
these
definitional
statements
and then
we see
it
added
them
into
utils
save
it
close
the
utils
readme
for this
repo
all right
let's
let's
just
quickly
let's
just
push
it
augmentation
push
then let's
read the
readme
in its
natural
habitat
okay

augmented
active
inference
readme
oh
interesting
okay
it was
empty
because it
wasn't
saved
in its
place
reload
the page
here it
is
okay
active
cursor
integration
the
integration
of active
inference
principles
with
cursor
ai
for
social
science
simulations
okay
no
clear it
up

we're
in
social
science
sim
make
all
needed
improvements
to
achieve
the
aims
of
readme
move
all
method
definitions
to
at
utils
dot
py
and
invoke
them
here
let's
see
it's
doing
this
this
is
a
I
believe

this
will
be
fixed
in
a
future
version
but
sometimes
it'll
be
like
going
all
the
way
back
up
the
repo
there
might
be
a
way
to
do
this
just
better
with
how
the
paths
are
handled
let's

just
see
what
happens
when
we
run
it
first
okay
cannot
import
name
agent
from
py
mdp
fix
this
but
we
know
that
it
has
access
to
py
mdp
methods
dot
markdown
so
this
is
the
one
that's

like
the
study
guide
for
what
py
mdp
does
now
for a
human
human
reading
this
document
as
mentioned
earlier
it's
probably
great
for
trains
long
car
rides
dark
rooms
all
these
kinds
of
places
where
you
might
want

to
read
something
just
really
pay
attention
to
it
but
it's
the
perfect
high
density
semantics
that
will be
helpful
here
let's
just
accept
whatever
it
did
see
if
it
is
better
and
if
not
we'll
give
it
a

little
help
no
module
named
pymdp
agent
so
it's
a
new
error
fix
this
so
let's
just
keep
on
finding
and
quashing
some
bugs
and
let's
just
see
if
can
we
get
to
a
pymdp
we
confusion
it's

like
no
no
I
wasn't
confused
you
don't
need
to
apologize
I
think
that
this
this
composer
functionality
again
super
cool
and
there's
also
probably
ways
to
use
it
a
lot
better
because
it
has
a
whole
another

view
layover
that
I'm
not
really
even
exploring
at
this
time
okay
it
cannot
import
utils
so
when
it's
one
kind
of
line
of
flight
and
just
like
quashing
bugs
that
seem
to
be
of
a
similar
ilk

I'll
keep
it
rolling
in
the
same
composer
but
then
when
it's
a
different
vector
we're
going
to
do
a
pivot
then
we
can
do
a
new
one
okay
no
module
pymdp
distributions
so
it's
kind
of
fixing

a
series
of
errors
related
to
accurately
calling
pymdp
submodules
this
could be
like
super
pymdp
file
structure
specific
but
nonetheless
that's
where we
are
okay
okay
okay
this
version
implements
a
simplified
version
of
simple
agent
using
only
numpy

we've
replaced
the
pymdp
specific
operations
with
basic
numpy
let's
see
what
it
looks
like
wow
like
so
then
it's
like
then
who
was
semantics
if
it
takes
the
pymdp
pomdp
semantics
you know
was it
all
what
okay
okay

okay
okay
got some
plotting
matlib
plot
errors
but i
can see
from the
top
okay
that it
that it
was doing
stuff
so
in
social
science
sim.py
save
all
terminal
logs
to a
file
log.txt
save
all
output
files
slash
visualizations
to a
folder
in
this

folder
output
slash
make
the
folder
if
not
there
claude
3.5
sonic
doing it
today
okay
accept
it
save
it
clear
it
clear
terminal
double
up
run
it
okay
got an
unexpected
argument
exit
here we
go
we have
agent
one
and

agent
zero
receiving
observations
updating
beliefs
taking
actions
amazing
see here
it just
put that
over here
so let's
look at
this one
this is in
the correct
location
this one
is
100
lines
long
this
one
is
it's
like
it's
seven
lines
long
just
implement
that
one
little

stub
but then
it did
it
in the
wrong
place
I don't
know
if
that's
I'm
just
using
it
weirdly
or
if
it's
just
getting
confused
because
we're
really
deep
in
this
nested
repo
if
you
just
do
one
project
per
repo

like
William
Blake
repo
cognitive
sovereignty
repo
then
the
embedding
is
faster
there's
less
context
but
that's
kind
of
the
delight
also
is
all
this
mixed
interactive
semantics
okay
clear
run it
again
we're
going to
get
the
probably
same

error
well
but
where did
it save
the logs
to
ensure
all
terminal
outputs
are
saved
in
this
folder
to
log
dot
txt
then
we'll
move
all
the
methods
also
just
for
context
here's
the
system
prompt
that I
give
cursor
use

professional
functional
modular
concise
elegant
interpretable
python
use
comprehensive
logging
to
terminal
and
all
other
programming
best
practices
I'm
going to
delete
that
part
because
maybe
it's
not
it's
over
learning
on
that
accept
save
it
clear
it
clear

it
run
it
okay
now
we got
to
this
area
fix
this
okay
we'll
progress
a little
bit
further
on
this
bring
in
a few
py
mdp
methods
till
then
if
anyone
wants
to
see
again
now
it's
editing
social
science

utilities
in
the
outer
folder
so
it's
like
why
but
if
anyone
writes
a live
chat
with
like
a
thought
or
a
question
we
can
definitely
do
it
okay
also
again
we know
that
we'd
be
able
to
bring
in

the
full
clone
in
py
mdp
and
then
just
explicitly
reference
it
and
that's
not
even
a
bad
strategy
but
let's
see
if
we
can
do
it
without
cloning
in
the
repo
so
move
all
methods
to
at

fills
dot
py
and
properly
here
okay
okay
accept
it
save
it
I'm
going to
copy
in
just
another
message
that
Andrew
wrote
read
it
here
FYI
with
appreciation
to the
work
being
done
by
both
the
SPM
and
py

dmp
which
is
heavily
based
on
SPM
I
can
confirm
via
hours
of
repo
cross
referencing
there's
no
easy
route
from
either
of
these
you
can't
get
there
from
here
packages
and
their
dependencies
to an
easy
ready to go
simulation

builder
already
simulating
two
agents
even
before
inspecting
the script
logic
awesome
big
feat
thank
you
awesome
Andrew
cool
okay
but
another way
we could
do this
would be
oh
simulation
results
okay
there's
three agents
they're taking
actions over
time
just by
calling
this
okay
now let's

see what
happens
let's just
re-index
cursor
make sure
that we're
spot
so here
it's
it's
all the
files that
we touched
it's
it needs
to re-index
it might
do a
little
mini
re-indexing
when you
call something
into local
context
but I
don't know
okay
bring it
into
the
composer
now
update
this
update
and

extend
these
methods
invoking
where
possible
at
pymdp
methods
so
that's
the one
that we've
been
looking at
and
enjoying
which is
it's like
the doc
strings
and the
input
output
for all
these
pymdp
functions
okay
let's
see how
it does
it's
it
drafts
it up
here
so here's

like where
the generation
is happening
and then
once it
finishes
this
v1
draft
then
it
runs
through
okay
and it
goes through
again
second
pass
it's kind
of like
measure
twice
cut
once
situation
okay
save it
clear
composer
social
agents
here we
see
abc
it's
like
qpi

this is
actually
doing
the
expected
free
energy
based
model
updating
for
a
and
b
okay
now
over
here
comprehensively
develop
I'm using
the
control
k
version
here
just
for
a
little
flavor
using
the
at
utils
dot
py
to

achieve
the
mission
of
003.2
maybe

there's
a way
to do
in the
system
prompt
like
just
say
and this
is also
where
programming
practice
and expertise
and all
this comes
into play
is like
maybe
maybe
there's a way
to make
sure like
that we
always call
the run
script
main
and then
we always

call the
utils
like utils
underscore
and then the
type of
utils
then we
could have
a system
prompt
that's
like
I
only
like
to run
scripts
called
dot
main
myself
and
please
ensure
that
utils
underscore
asterisk
are
comprehensive
and
you know
make it
so that
Carl
Friston
would smile

upon it
clear
social
science
sim
okay
can't
import
name
control
new
errors
fix it
script
overview
meanwhile
while the
control
i is
going
control
k

develop
this
given
all
we
know
now
accept
whatever
it
says
clear
run
it
again

reset
that
one's
done
accept
it
pop
back
to
social
science
sim
can't
import
inference
new
one
fix
it
meanwhile
back
to
3.2
develop
this
given
what
we
know
now
okay
see
here
now
utils
again
is going
to

the
wrong
place
we'll
get it
there
let's
just see
what it
does
okay
okay
create
create
contribution
guidelines
for potential
collaborators
set up a
project wiki
for detailed
documentation
tutorial
consider
presenting the
project at
relevant
conferences or
meetups
like the
fourth applied
active inference
symposium
from november
13th to
15th
2024
possibly

let's just
say
develop
section
nine
comprehensively
comma
absurdly
comma
professionally
then we'll
pop back to
the simulator
okay
okay okay
okay
all right
it's all about
nine now
wow
okay
talk about
cart before
the horse
it's like
why
why not
just
focus on
section
nine
aren't you
interested in
like
subsections
reject it
it's okay
we know

it would
okay
okay
okay
Andrew wrote
typically
infer policies
and sample
action
functions
are called
separately
albeit
often
adjacently
as pi
mvp
the
lom
seemed to
realize
it could
combine
them
into a
single
function
flow
okay
let me
say
now
given
at
so
and then
Andrew wrote
subject to

change
of course
point is
fascinating
to see
how the
lom
might
make
optimizations
not
inherent
to the
original
libraries
I mean
yes
that's
that's
very
characteristically
understated
yes
it suggests
stuff that's
like
it's an
adjacency
in like
dimension
754
but
for a
given
person
it's like
it might
pop up

to your
awareness
in a
moment
it might
not
but even
if
what popped
up in
your
awareness
were like
awesome
epic
professional
modular
absurd
great
every
single
second
or you're
doing it
10 times
a second
it's like
you still
might never
just riff
enough
let alone
capture
and do
that kind
of cosmic
fishing
explain

this
comprehensively
okay so
now wall
control
L
is going
reset it
let's add
in the
utils
run it
but we know
it's going to
give an error
add it to the
composer
fix this
with reference
slash
modification
only
to the
local
folder
okay
combining
infert
policy
sample
action
functions
into a
single
function
or flow
can
streamline

the
simulation
process
clear it
run it
no
module
pymdp
at
aldos
fix this
it's making
a lot of
changes
so it's
like again
there's
there's so
many ways
to do
the
professional
programming
this is
just one
sort of
pythonic
strategy
okay now
it's bringing
in pymdp
these
units
save it
clear it
double up
run
can't import

name
inference
fix it
also it's
like we
could have
just added
well it's
it can't
import
inference
maybe
inference
doesn't
even
exist
that's
those are
a few
of the
situations
see now
it might
just be
kind of
floundering
right here
let's
see
no
module
pymdp
dot core
let's just
go over
to pymdp
and just
see what

they are
but this
also
really
this is
getting
pretty
specific
into pymdp's
folder and
sub
module
structure
yeah
just maybe
10 to
30 more
minutes
but if
people have
comments or
questions
please just
go for it
okay so
there it's
just saying
you know
what
ignore that
core
mention
run it
okay
cannot
import
inference
and delete

it
cannot
import
control
we
let's just
accept
importing
pymdp
period
fix

this
however
yeah
okay
Andrew wrote
inference
algos
etc
do
exist
but the
dependency
structure is
highly
complex
yeah
okay
okay
okay
okay
okay
let's
let's do a
little
let's do some
radical pymdp

surgery
here
okay
cd double dot
pop up a
level
ls
that's what's
there
cd double dot
pop up a
level
ls
there's better
ways to do
this
I'm just
showing one
way to do
it
cd
active
inference
now we're
in active
inference
it's not the
augmented one
okay
reload our
llms
get them
ready
okay
python 3
pymdpeople
clone in
pymdp

okay
okay
here we
go
new script
this is going
to be called
flat pymdp
pymdp
underscore
flatten
dot
py
okay
here
given all
the functions
in
pymdp
package
iterate
over the
entire
python
package
write a
script
that will
iterate
over the
entire
python
package
pulling
out
all
python
method

slash
entity
definitions
e.g.
across
and through
modules
sub
folders
etc

and
output
a
mega
I won't
use that
output
a
text
file
pymdp
methods
flat
dot
py
which
let's do
pymdp
utils
pymdp
all
utils
which
contains
verbatim
all

pymdp
methods
for the
calling
okay
so we're
going to
generate
the script
here
that's going to
flatten pymdp
okay
again
it pulled
it up
here
copy it
pymdp
flatten
paste it
in
okay
okay
like
if
this works
on one
shot
okay
there's
pymdp
flatten
python
3
pymdp
underscore
flatten

okay
what do
you think
extract
definitions
ast
walk
process
package
main
all
right
okay
okay
a little bit of weirdness just because of
i o operations on closed file
that could be just due to like something weird about how i opened the
okay
extracted 24 definitions
all right
all right
looking good
ensure that this
script
iterates through all folders
and sub
folders
within
dot
slash
pymdp
now
it
let's just see
os walk
accept it
save it
clear it

double up
run it
24 definitions
still i am only getting 24
definitions
there should be
hundreds
now
this
even if we get to hundreds
it won't be the right output
right
because like
there's going to be a method defined
in utils
and i expect there'll be a similar one in
slash
jacks
slash utils
okay
clear
double up
run it
okay
okay
look
in this
very folder
comma
my colleague
there are
sub
folders
in what was
cloned
by
pymdp
pull

that output
there
is the whole
pymdp
script
that's
what we
need the whole team
to look through
so we can get all
hundreds of methods
to make
a
flattened
single
mega
utils
file
for simpler
referencing
remember
we're just
next
mates
the script
is looking at the
installed
pymdp
package
instead of
locally
cloned
repo
there we go
but why didn't you just say so
now
there we go
456

to all
utils
okay
now I'm going to pull this back
into
the scripts
well done
cursor
take a quick breather
okay
clear
cd double dot
pull up
cd
augmented
enter
ls
cd
scripts
ls
go over to
cursor
double check
it's all
re-indexed
delete the original
pymdp
there it is
new files
we'll just make sure
it gets them all
400
456
definition
that says
andrew
incroyable
that's like

exactly how I feel
it's like
what
what
oh for abjus
incroyable
day
okay
social science
sim
add in
to this
context
window
pymdp
underscore
all
utils
develop
this
multi-agent
social
simulation
given
all
methods
fully
defined
in
pymdp
underscore
all
utils
dot py
in this
folder
okay
meanwhile

well let's
let's let it
edit
then
then
we'll
okay
okay
it's kind
of writing
it in
the composer
window
again
this is
this might
just be a
tiny
cursor
bug
it'll be
like
there it
is
there's
my answer
but it's
like
well
no
let's
or
so it
doesn't
suggest
a code
edit
suggest

all
those
code
edits
to
social
science
sim
it's like
oh
I'm sorry
yep
pretty cool
okay
now
this is
where
it's
doing
the
composer
mode
correctly
it
opens
up
a
second
tab
then
it
writes
all
those
code
edits
there
and

then
it'll
rescan
this one
and say
well
this was
my
updated
draft
and
this
was
the
original
draft
so
now
pulls
in
let's
see
how
this
does
fix
this
remove
the
need
to
ask
about
version
if
that
is
simpler

okay
utils.py
it's
because
it's
because
it's
calling
both
the
utils
and
well
let's
just
rename
this
one
utils
yeah
rename
this
utils
well
first
let's
delete
the
ones
we
had
we
if
we
call
it
utils
old

it
might
get
confused
okay
utils.py
then
social
science
sim
utils
import
star
okay
fix
this
removing
the
need
for
the
method
or
invoking
it
or
writing
it
if
it
is
missing
but
then
here's
the
deal
utils.py

which is
the
flattened
all
it's
20,000
lines
long
so
then
it'll
have
a
hard
time
reaching
in
and
making
the
targeted
edit
to
something
in
here
see
it
might
just
be
like
oh
we
don't
need
that
one

let's
see
it
might
just
be
like
let's
delete
19.9
out of
the
20,000
lines
okay
a
file
too
long
see
edit
command
is
limited
to
files
at
most
around
400
lines
long
so
then
it
just
cancels
it

so
I
I
I
I
get
that
I
get
that
all
pyMDP
utils
social
science
sim
from
all
pyMDP
utils
save
it
write
this
so

so
that
it
only
uses
functions
defined
in
at
all

pyMDP
utils

it imports
everything
which is
perfect
don't change
that
use the
pyMDP
methods
to
make
this
multi
agent
simulation
that
initializes
simulates
and
visualizes
a social
multi
agent
setting
we'll do a few
more minutes
if anyone has
comments or
questions
let's
see
that
also
again
it's

defining
methods
here
so
see
what it
does
social
agent
now
uses
agent
from
pyMDP
along
with
random
a
b
etc
ok
ok
remove
the need
for
unit
test
not
sure
what
that
is
we
can
like
look
that's
the

unit
testing
methods
I
understand
see
but like
do you
like
philosophically
do you
let's
just ask
it
not that
what it
says
will be
an
answer
philosophically
would
you
say
that
you
fully
understand
this
script
and
all
it
implies
kind of
a trick
question
because

it's
like
all
it
implies
okay
accept
it
clear
it
double
run
it
okay
let's
try it
here's
what it
says
did
you
mean
okay
yes
I did
mean
python
3
okay
so
okay
well
there's
the answer
for
whether
it
thinks

it
understands
it
but
that's
like
that's
like
something
for
people
to
consider
like
this is
why
I
leave
the
same
composer
window
running
really
make
sure
that
social
science
sim
does
not
need
any
unit
test
stuff
at

all
okay
but
meanwhile
new
side
chat
just
dropped
write

a
hilarious
age
appropriate
esoteric
lore
laden
who's
on
first
style
dialogue
for
fully
explaining
allegorically
to

someone
familiar
with
reinforcement
learning
everything
happening
in this
active

inference
multi
agent
simulation
ensure
many
baseball
history
historical
references
comma
as well
okay
update
this
so that
it
runs
with
just
python
3
social
science
sim
dot
py
no
command
line
arguments
I think I
accidentally
terminated
the
let's
do
sonnet

I think
it does
sometimes
funnier
who's
on
first
dialogue
okay
we have
two
two
sonnets
working
for us
here
we got
the
baseball
dialogue
slow
position
12
okay
okay
out of the
command line
arguments
now we
can call
it the
way we
want
accept
it
save
it
reset

it
clear
it
double
up
run
it
what did
I say
sonnet
ensure
that
unit
test
is not
required
at
all
I
never
this
is
something
I've
never
said
before
I
never
want
to
see
that
bug
again
meanwhile
who's
on

first
dot
md
okay
okay
okay
okay
okay
let's
not get
ahead
of
ourselves
save
it
clear
it
double
up
fix
it
and
remember
what I
said
last
time
we'll
see if
we can
do
like
a couple
more
see if
we
fix
it

and
then
otherwise
we'll
close
with
reading
the
who's
on
first
summary
and
then
at
the
end
of
that
I
will
read
any
comments
that
people
have
or
questions
that
they
have
asked
in
the
meanwhile
and
that

will
conclude
in
for
ant
stream
three
no no
you don't
have to
apologize
I
just
said
remember
what
I
said
I
let's
just
see
if
cursor
has
self
awareness
fix
fix
this
in
a
way
where
you
cursor
don't
give

me
that
error
okay
let's
see how
that
goes
okay
ctrl
shift
l
Andrew
wrote
restructuring
a library
that's
taken years
of development
and various
programmers
to develop
not to
mention
everything
contributed
to its
SPM
precursor
can it
be done
in a few
sittings
and I'll
say
that's
what
my

oh
see
file
too
long
to
avoid
cursor
I'll
make
it
more
so
we'll
leave
it
there
we'll
push
the
update
so
updates
not all
fully
working
the
semantics
are
basically
there
though
for
any
and all
to build
upon
and help

through
okay
we'll push
that
structure
this
for
professional
markdown
viewing
that's
what
my
friend
Andrew
said
prepare
a total
response
to him
that is
respectful
and age
appropriate
okay
we're going
to read
the response
to Andrew
then we're
going to
push
Andrew
I appreciate
your perspective
on this
you're
absolutely

right
that
restructuring
a library
with years
of development
history
and contributions
from multiple
programmers
is significant
undertaking
it's not
something to be
taken lightly
or rushed
through
dot dot dot
dot dot dot
dot dot
dot
what are your
thoughts
this is kind
of like
hold on
there
partner
slow down
restructure
PyMDP
by flattening
it and
squeezing
it
okay
close it
save it

close it
we did
what we
did
save it
close it
clear it
close the
terminal
clear it
again
close it
goodbye
cursor
for today
who is
on
first
question
in
tarot
bang
then we'll
read it
there
and then
I'll
look at
any live
chats
augmented
docks
okay
here we
go
it
always
does

the
27
Yankees
okay
hey
Costello
have you
heard about
this new
active
inference
multi-agent
simulation
it's like
the
1927
Yankees
you mean
with
Babe Ruth
and
Lou Gehrig
no no
I'm talking
about
social agent
and
its
teammates
they're
like the
murderer's
row of
artificial
intelligence
wait
who's
on

first
in
this
simulation
no
who's
not
in
this
simulation
we got
social agent
on first
observation
at second
and action
at third
okay
so who's
social agent
exactly
social agents
on first
just like
our old
pal
who
I'm
asking
you
who's
social agent
no
no
social agent
is like
Babe Ruth
he's the

heavy hitter
of the
simulation
he's got
a random
A matrix
and B
matrix
just like
the
Bambino
had his
legendary
bat
and glove
so
social agent
has a
bat
and glove
no
he's got
beliefs
and
policies
the
A matrix
is his
belief
about how
observations
relate to
hidden
states
like how
the
babe
could read

a
picture
the
B
matrix
is his
belief
about
how
actions
change
the
world
like
how
Ruth
knew
just
where
to
swing
to
hit
it
out
of
!
action

is
like
Tony
Lazeri
the
Yankees
clutch
hitter

when
social agent
needs to
make a
move
action
steps
up to
the
plate
so
how do
they play
together
well
it's like a
perfectly executed
double play
a triple play
social agent
uses his A and B
matrices to
infer the best
action
just like
Ruth sizing up
a pitch
then action
takes that
inference
and makes a
move
like Lazeri
turning a
double play
meanwhile
observation
is constantly

updating the
game state
like combs
tracking
fly balls
in the
outfield
but what's
the goal
of this
game
huh
that's
where it
gets
interesting
the goal
is to
maximize
expected
free
energy
which is
like
trying
to win
the
world
series
of
cognition
it's
all about
minimizing
surprise
and
uncertainty
just

like
how
the
27
Yankees
minimized
their
opponents
chances
of
winning
so
they're
trying
to
predict
what's
going
to
happen
exactly
it's
like
they're
playing
interdimensional
chess
while
everyone
else
is
playing
checkers
they're
not
just
reacting
to

the
environment
they're
actively
trying
to
shape
it
just
like
how
Babe
Ruth
called
his
shots
this
is
making
my
head
spin
more
than
a
Dizzy
Dean
curveball
okay
that's
hilarious
don't
worry
Costello
just
remember
in
this

game
every
agent
is
trying
to
be
the
Babe
Ruth
of
their
own
cognitive
world
series
they're
not
just
learning
from
rewards
like
in
old
school
reinforcement
learning
they're
actively
inferring
the best
way
to
play
the
game
of

life
I
think
I
need
to
go
sit
in
the
bullpen
and
think
about
this
for
a
while
that's
the
spirit
just
like
how
active
inference
agents
are
always
updating
their
models
you're
updating
your
understanding
you'll
be

a
cognitive
hall
of
!
time

Conclusion
This
highlights
key components
such as
the
social
agent
observation
action
as well
as
core
concepts
like
belief
updating
and
free
energy
minimization
all
right
thanks
everyone
for
watching
that
was
super
fun

hopefully
you
enjoyed
some
of
these
methods
and
demonstrations
!
It's
all
happening
at the
Active
Inference
Institute
slash
Active
Inference
repo
everything
we looked
at is
open
sourced
and
pushed
it
would
be
awesome
for
anyone
to
use
play
extend

further
come
participate
have
fun
alright
thanks
bye
!